🌐 TEMA 05 • HTTP na Prática

O que realmente viaja do navegador para o servidor: método, cabeçalhos e corpo da requisição, observados no DevTools

🏞️ Anatomia HTTP 📮 GET × 📮 POST 🔬 DevTools 🧪 httpbin.org

🧭 Desafio investigativo

🏞️ 1. Anatomia HTTP | 🧪 2. Experimento | 🔬 3. DevTools passo a passo | 📊 4. Comparação

🧭 Desafio investigativo

Depois de executar os dois testes e observar no DevTools, responda às cinco perguntas abaixo. Cada uma tem um botão que revela o gabarito.

1 **Qual parte da requisição mudou quando trocamos GET por POST?**

a principal mudança foi onde os dados do formulário vão. No GET, os dados aparecem na URL após o ?. No POST, os dados vão no corpo da requisição, e a URL fica limpa. Além disso, o POST ganha os cabecalhos Content-Type e Content-Length, porque há um corpo

🙈 Ocultar gabarito

> **Resposta:** a principal mudança foi *onde os dados do formulário vão*. No GET, os dados aparecem na URL após o `?`. No POST, os dados vão no **corpo** da requisição, e a URL fica limpa. Além disso, o POST ganha os cabeçalhos `Content-Type` e `Content-Length`, porque há um corpo a descrever.

2 **Em qual teste o nome "Ana" apareceu na URL?**

no teste com GET. A URL ficou parecida com https://httpbin.org/get?nome=Ana&email=ana%40exemplo.com. No POST, a URL continuou apenas https://httpbin.org/post, sem parâmetros visíveis.

🙈 Ocultar gabarito

Resposta: no teste com **GET**. A URL ficou parecida com `https://httpbin.org/get?nome=Ana&email=ana%40exemplo.com`. No POST, a URL continuou apenas `https://httpbin.org/post`, sem parâmetros visíveis.

3 Qual cabeçalho informa o formato dos dados enviados?

o Content-Type. No POST com formulário comum, seu valor é application/x-www-form-urlencoded, indicando que os dados estão codificados no formato chave=valor separados por &. Se o formulário enviasse arquivo, seria multipart/form-data.

🙈 Ocultar gabarito

Resposta: o `Content-Type`. No POST com formulário comum, seu valor é `application/x-www-form-urlencoded`, indicando que os dados estão codificados no formato chave=valor separados por `&`. Se o formulário enviasse arquivo, seria `multipart/form-data`.

4 No POST, em qual painel do DevTools encontramos o nome enviado?

na seção Payload (pode aparecer como Request Payload ou Form Data, dependendo do navegador). Ali estão listados os campos nome: Ana e email: ana@exemplo.com, exatamente como foram digitados e enviados no corpo da requisição.

🙈 Ocultar gabarito

Resposta: na seção **Payload** (pode aparecer como *Request Payload* ou *Form Data*, dependendo do navegador). Ali estão listados os campos `nome: Ana` e `email: ana@exemplo.com`, exatamente como foram digitados e enviados no corpo da requisição.

5 Se os dois métodos têm cabeçalhos, por que não podemos dizer que somente o POST utiliza headers?

porque os cabeçalhos HTTP fazem parte de toda mensagem HTTP, independentemente do método. GET, POST, PUT, DELETE, PATCH — todos enviam cabeçalhos. O que difere é o conteúdo dos cabeçalhos: POST costuma ter Content-Type e Content-Length porque

🙈 Ocultar gabarito

Resposta: porque os cabeçalhos HTTP fazem parte de *toda* mensagem HTTP, independentemente do método. GET, POST, PUT, DELETE, PATCH — todos enviam cabeçalhos. O que difere é o *conteúdo* dos cabeçalhos: POST costuma ter `Content-Type` e `Content-Length` porque envia corpo; GET não costuma ter, porque não envia. Mas os cabeçalhos básicos (`Host` , `User-Agent` , `Accept`) aparecem sempre.

💡 **Extensão sugerida:** depois de responder às perguntas, abra a **aba Response** (ou *Preview*) da requisição POST no DevTools e compare o JSON do `httpbin.org/post` com o do `httpbin.org/get` . No GET, os dados aparecem em `args` . No POST, aparecem em `form` . Essa comparação fecha o ciclo completo: *formulário → requisição HTTP → servidor → resposta*.